

TRAVAUX PRATIQUES

Protocole MCP avec LangChain

Model Context Protocol — Serveurs, Agents & Middleware

Realise par : Labrazi mohammed yassine et Sara slimane

Technologies : Python, FastMCP, LangChain, LangGraph

LLM : OpenRouter — GPT-OSS 120B

Table des matières

1	Introduction & Objectifs pédagogiques	2
1.1	Prérequis et installation	2
2	Exercice 1 — Premier serveur MCP Niveau 1/3	2
2.1	Contexte	2
2.2	Outils exposés	2
2.3	Code source — <code>math_server.py</code>	2
2.4	Test avec l’Inspector	4
3	Exercice 2 — Premier agent consommateur Niveau 2/3	4
3.1	Contexte	5
3.2	Code source — <code>math_agent.py</code>	5
3.3	Trace d’exécution attendue	6
4	Exercice 3 — Multi-serveurs et Middleware Niveau 3/3	7
4.1	Contexte	7
4.2	Code source — <code>hr_server.py</code>	7
4.3	Code source — <code>notes_server.py</code>	8
4.4	Code source — <code>dual_agent.py</code>	9
4.5	Trace d’exécution attendue	11
5	Pour aller plus loin (optionnel)	11
6	Modalités de rendu	11
	Récapitulatif des concepts clés	12

1 Introduction & Objectifs pédagogiques

Ce TP a pour objectif de maîtriser le **Model Context Protocol (MCP)**, un protocole ouvert permettant d'exposer des fonctions Python comme outils consommables par des agents LLM. À l'issue de ces travaux pratiques, vous serez capable de :

- Écrire un serveur MCP en Python avec **FastMCP** et le décorateur `@mcp.tool()`.
- Tester un serveur MCP de manière isolée avec l'inspecteur officiel.
- Connecter un agent LangChain à un serveur MCP via `MultiServerMCPClient`.
- Comprendre la différence entre invocation synchrone et asynchrone.
- Composer plusieurs serveurs MCP dans un seul agent.
- Ajouter un middleware de journalisation sur des outils MCP.

1.1 Prérequis et installation

Terminal

```
# Ajouter les paquets MCP à l'environnement
uv add mcp langchain-mcp-adapters

# Vérifier l'installation
python -c "import mcp; print('mcp', mcp.__version__)"
python -c "import langchain_mcp_adapters; print('adapters OK')"
```

Le fichier `.env` du chapitre précédent doit être présent à la racine du projet (il contient `OPENAI_API_KEY` pour OpenRouter).

2 Exercice 1 — Premier serveur MCP Niveau 1/3

2.1 Contexte

Vous allez créer un serveur MCP minimal qui expose des outils de calcul arithmétique. L'objectif est de comprendre la structure d'un serveur FastMCP et le rôle des docstrings avant d'aborder un cas métier plus complexe.

2.2 Outils exposés

Outil	Signature	Description
add	add(a: float, b: float) -> float	Additionne deux nombres et retourne la somme.
multiply	multiply(a: float, b: float) -> float	Multiplie deux nombres et retourne le produit.
divide	divide(a: float, b: float) -> float	Divise a par b ; message d'erreur si b=0.

2.3 Code source — `math_server.py`

```
1 """
2 Exercice 1 -- Premier serveur MCP
3 Serveur FastMCP exposant des outils de calcul arithmetique.
4 """
5 from mcp.server.fastmcp import FastMCP
6
7 mcp = FastMCP("math-server")
8
9
10 @mcp.tool()
11 def add(a: float, b: float) -> float:
12     """
13     Adds two numbers and returns their sum.
14     Use this tool whenever you need to perform an addition.
15
16     Args:
17         a: The first operand.
18         b: The second operand.
19     Returns:
20         The sum of a and b as a float.
21     Example:
22         add(3, 5) -> 8.0
23         add(1.5, 2.5) -> 4.0
24     """
25     result = a + b
26     print(f"[math_server] add({a}, {b}) = {result}")
27     return result
28
29
30 @mcp.tool()
31 def multiply(a: float, b: float) -> float:
32     """
33     Multiplies two numbers and returns their product.
34
35     Args:
36         a: The first factor.
37         b: The second factor.
38     Returns:
39         The product of a and b as a float.
40     Example:
41         multiply(7, 6) -> 42.0
42         multiply(2.5, 4) -> 10.0
43     """
44     result = a * b
45     print(f"[math_server] multiply({a}, {b}) = {result}")
46     return result
47
48
49 @mcp.tool()
50 def divide(a: float, b: float) -> float | str:
51     """
52     Divides a by b. Returns an error message if b == 0.
53
54     Args:
55         a: The dividend.
56         b: The divisor (must not be zero).
57     Returns:
```

```

58         The quotient a / b, or an error message string.
59     Example:
60         divide(10, 2)  -> 5.0
61         divide(10, 0)  -> "Error: division by zero is undefined."
62     """
63     if b == 0:
64         msg = "Error: division by zero is undefined."
65         print(f"[math_server] divide({a}, {b}) -> {msg}")
66         return msg
67     result = a / b
68     print(f"[math_server] divide({a}, {b}) = {result}")
69     return result
70
71
72 if __name__ == "__main__":
73     mcp.run(transport="stdio")

```

Listing 1 – math_server.py – Serveur FastMCP de calcul arithmétique

2.4 Test avec l'Inspector

Terminal

```
npx @modelcontextprotocol/inspector python math_server.py
```

L'inspector ouvre une page web. Vérifiez que :

1. Les outils `add`, `multiply` et `divide` apparaissent dans la liste.
2. Les docstrings sont affichées correctement dans le panneau de droite.
3. L'appel `add(3, 5)` retourne 8.
4. L'appel `multiply(7, 6)` retourne 42.

Conseil de débogage

- Vérifiez l'import : `from mcp.server.fastmcp import FastMCP` (pas `mcp.fastmcp`).
- Vérifiez la présence du bloc `if __name__ == "__main__": mcp.run(transport="stdio")`.
- Si `npx` échoue : installez Node.js (≥ 18) ou utilisez l'inspector via `uvx`.

Critères de validation

- Le fichier `math_server.py` démarre sans erreur.
- Les trois outils sont visibles dans l'inspector avec leurs docstrings.
- Les appels manuels retournent les résultats corrects.
- Bonus :** `divide(10, 0)` retourne un message d'erreur sans planter le serveur.

3 Exercice 2 — Premier agent consommateur Niveau 2/3

3.1 Contexte

Vous allez créer un agent LangChain qui se connecte au serveur `math_server.py`. L'objectif est de maîtriser `MultiServerMCPClient` et le mode asynchrone obligatoire avec `langchain-mcp-adapters`.

3.2 Code source — `math_agent.py`

```

1  """
2  Exercice 2 -- Premier agent consommateur MCP
3  Agent LangChain connecte a math_server.py via MultiServerMCPClient.
4  """
5  import asyncio
6  import os
7  from pathlib import Path
8  from dotenv import load_dotenv
9  from langchain_openai import ChatOpenAI
10 from langchain_mcp_adapters.client import MultiServerMCPClient
11 from langgraph.prebuilt import create_react_agent
12
13 load_dotenv()
14
15 llm = ChatOpenAI(
16     model="openai/gpt-oss-120b:free",
17     openai_api_key=os.getenv("OPENAI_API_KEY"),
18     openai_api_base="https://openrouter.ai/api/v1",
19 )
20
21 SERVER_PATH = str(Path(__file__).parent / "math_server.py")
22
23
24 async def main():
25     async with MultiServerMCPClient(
26         {
27             "math": {
28                 "command": "python",
29                 "args": [SERVER_PATH],
30                 "transport": "stdio",
31             }
32         }
33     ) as client:
34         # Decouverte des outils
35         tools = await client.get_tools()
36         tool_names = [t.name for t in tools]
37         print(f"\nOutils decouverts ({len(tools)}) : {tool_names}\n")
38
39         # Creation de l'agent ReAct
40         agent = create_react_agent(model=llm, tools=tools)
41
42         # Question principale
43         question = "Calcule (17 + 23) x 4"
44         print(f"Question : {question}\n")
45         response = await agent.ainvoke(
46             {"messages": [{"role": "user", "content": question}]}
47         )
48         print(f"\nReponse finale :
          {response['messages'][-1].content}\n")

```

```

49
50     # Bonus : test de divide
51     print("-" * 50)
52     print("Bonus -- Test de l'outil divide :\n")
53     for q in ["Calcule 10 divise par 2.", "Calcule 10 divise par
54               0."]:
55         print(f"Question : {q}")
56         resp = await agent.ainvoke(
57             {"messages": [{"role": "user", "content": q}]}
58         )
59         print(f"Reponse  : {resp['messages'][-1].content}\n")
60
61 if __name__ == "__main__":
62     asyncio.run(main())

```

Listing 2 – math_agent.py – Agent LangChain connecte au serveur MCP

3.3 Trace d'exécution attendue

Terminal

```

Outils decouverts (3) : ['add', 'multiply', 'divide']
Question : Calcule (17 + 23) x 4
[math_server] add(17, 23) = 40
math_server
multiply(40, 4) = 160
Reponse finale : Le resultat de (17 + 23) x 4 est 160.

-----
Bonus - Test de l'outil divide :
Question : Calcule 10 divise par 2.
math_server
divide(10.0, 2.0) = 5.0
Reponse : 10 divise par 2 est egal a 5.
Question : Calcule 10 divise par 0.
math_server
divide(10.0, 0.0) -> Error: division by zero is undefined.
Reponse : La division par zero est impossible.

```

Pièges courants

- Oublier `await` devant `client.get_tools()` : le programme se termine silencieusement.
- Utiliser `invoke()` au lieu de `ainvoke()` : avec MCP, tout doit être asynchrone.
- Dans un script `.py`, le point d'entrée doit être `asyncio.run(main())`.

Criteres de validation

L'agent appelle `add` puis `multiply` (deux `tool_calls` successifs visibles).
La réponse finale est correcte : **160**.

Bonus : `divide(10, 0)` retourne un message d'erreur sans planter.

4 Exercice 3 — Multi-serveurs et Middleware Niveau 3/3

4.1 Contexte

Vous allez connecter un agent à deux serveurs MCP simultanément et ajouter un middleware de journalisation. Cet exercice simule une architecture d'entreprise réelle : un serveur **RH** existant et un nouveau service **notes** cohabitant dans le même agent.

4.2 Code source — `hr_server.py`

```

1  """
2  hr_server.py -- Serveur MCP RH
3  Expose trois outils : get_employee, list_employees_by_department,
4  update_salary.
5  """
6  from mcp.server.fastmcp import FastMCP
7
8  mcp = FastMCP("hr-server")
9
10 EMPLOYEES = {
11     "E001": {"name": "Alice Martin", "department": "Engineering",
12             "salary": 72000},
13     "E002": {"name": "Bob Dupont", "department": "Engineering",
14             "salary": 68000},
15     "E003": {"name": "Karim Bennani", "department": "Sales",
16             "salary": 55000},
17     "E004": {"name": "Sofia Reyes", "department": "Sales",
18             "salary": 57000},
19     "E005": {"name": "Lena Muller", "department": "HR",
20             "salary": 61000},
21 }
22
23 @mcp.tool()
24 def get_employee(employee_id: str) -> dict | str:
25     """
26     Retrieves the full record of an employee by their ID.
27
28     Args:
29         employee_id: The unique employee identifier (e.g. 'E001').
30     Returns:
31         A dict with employee data, or an error message string.
32     """
33     emp = EMPLOYEES.get(employee_id.upper())
34     if emp is None:
35         return f"Error: employee '{employee_id}' not found."
36     print(f"[hr_server] get_employee({employee_id}) -> {emp}")
37     return {"id": employee_id.upper(), **emp}
38
39 @mcp.tool()

```

```

36 def list_employees_by_department(department: str) -> list[dict]:
37     """
38     Returns all employees belonging to the specified department.
39
40     Args:
41         department: The department name (case-insensitive).
42     """
43     result = [
44         {"id": eid, **data}
45         for eid, data in EMPLOYEES.items()
46         if data["department"].lower() == department.lower()
47     ]
48     print(f"[hr_server] list_by_dept({department!r}) -> {len(result)} employees")
49     return result
50
51
52 @mcp.tool()
53 def update_salary(employee_id: str, new_salary: float) -> dict | str:
54     """
55     Updates the salary of the specified employee.
56
57     Args:
58         employee_id: The unique employee identifier.
59         new_salary: The new annual salary (must be positive).
60     """
61     eid = employee_id.upper()
62     if eid not in EMPLOYEES:
63         return f"Error: employee '{employee_id}' not found."
64     if new_salary <= 0:
65         return "Error: salary must be a positive number."
66     old = EMPLOYEES[eid]["salary"]
67     EMPLOYEES[eid]["salary"] = new_salary
68     print(f"[hr_server] update_salary({eid}) {old} -> {new_salary}")
69     return {"id": eid, **EMPLOYEES[eid]}
70
71
72 if __name__ == "__main__":
73     mcp.run(transport="stdio")

```

Listing 3 – hr_server.py – Serveur MCP RH (copie depuis la section 10.7 du cours)

4.3 Code source — notes_server.py

```

1  """
2  Exercice 3 -- Serveur MCP de notes
3  Expose deux outils : add_note et get_notes.
4  Stockage en memoire (dictionnaire Python).
5  """
6  from mcp.server.fastmcp import FastMCP
7
8  mcp = FastMCP("notes-server")
9
10 # {employee_id: [note1, note2, ...]}
11 NOTES: dict[str, list[str]] = {}
12
13

```

```

14 @mcp.tool()
15 def add_note(employee_id: str, note: str) -> str:
16     """
17     Attaches a text note to an employee's file.
18
19     Args:
20         employee_id: The employee identifier (e.g. 'E003').
21         note:         The text note to add.
22     Returns:
23         A confirmation message.
24     """
25     eid = employee_id.upper()
26     if eid not in NOTES:
27         NOTES[eid] = []
28     NOTES[eid].append(note)
29     print(f"[notes_server] add_note({eid!r}, {note!r})")
30     return f"Note added to {eid}: '{note}'"
31
32
33 @mcp.tool()
34 def get_notes(employee_id: str) -> list[str]:
35     """
36     Returns the list of notes attached to an employee's file.
37     Returns an empty list if no notes exist.
38
39     Args:
40         employee_id: The employee identifier (e.g. 'E003').
41     Returns:
42         A list of note strings, possibly empty.
43     """
44     eid = employee_id.upper()
45     notes = NOTES.get(eid, [])
46     print(f"[notes_server] get_notes({eid!r}) -> {notes}")
47     return notes
48
49
50 if __name__ == "__main__":
51     mcp.run(transport="stdio")

```

Listing 4 – notes_server.py – Serveur MCP de gestion de notes

4.4 Code source — dual_agent.py

```

1  """
2  Exercice 3 -- Agent multi-serveurs avec middleware de journalisation.
3  Connecte simultanément a hr_server.py et notes_server.py.
4  """
5  import asyncio
6  import os
7  from pathlib import Path
8  from functools import wraps
9  from dotenv import load_dotenv
10 from langchain_openai import ChatOpenAI
11 from langchain_mcp_adapters.client import MultiServerMCPClient
12 from langgraph.prebuilt import create_react_agent
13
14 load_dotenv()

```

```

15
16 llm = ChatOpenAI(
17     model="openai/gpt-oss-120b:free",
18     openai_api_key=os.getenv("OPENAI_API_KEY"),
19     openai_api_base="https://openrouter.ai/api/v1",
20 )
21
22 BASE = Path(__file__).parent
23
24
25 def logger_middleware(tool_func):
26     """Middleware qui journalise chaque appel d'outil MCP."""
27     @wraps(tool_func)
28     async def wrapper(*args, **kwargs):
29         print(f"[MCP] tool={tool_func.name} args={kwargs}")
30         return await tool_func(*args, **kwargs)
31     return wrapper
32
33
34 async def main():
35     async with MultiServerMCPClient(
36         {
37             "hr": {
38                 "command": "python",
39                 "args": [str(BASE / "hr_server.py")],
40                 "transport": "stdio",
41             },
42             "notes": {
43                 "command": "python",
44                 "args": [str(BASE / "notes_server.py")],
45                 "transport": "stdio",
46             },
47         }
48     ) as client:
49         tools = await client.get_tools()
50         tool_names = [t.name for t in tools]
51         print(f"\nOutils decouverts ({len(tools)}) : {tool_names}\n")
52
53         # Application du middleware
54         wrapped_tools = [logger_middleware(t) for t in tools]
55         agent = create_react_agent(model=llm, tools=wrapped_tools)
56
57         question = (
58             "Recupere la fiche de E003 puis ajoute la note "
59             "'Excellent trimestre' a son dossier."
60         )
61         print(f"Question : {question}\n")
62         response = await agent.ainvoke(
63             {"messages": [{"role": "user", "content": question}]}
64         )
65         print(f"\nReponse finale :
66             {response['messages'][-1].content}\n")
67
68 if __name__ == "__main__":
69     asyncio.run(main())

```

Listing 5 – dual_agent.py – Agent dual avec middleware de journalisation

4.5 Trace d'exécution attendue

Terminal

```
Outils decouverts (5) : ['get_employee', 'list_employees_by_department',
                        'update_salary', 'add_note', 'get_notes']

Question : Recupere la fiche de E003, ajoute 'Excellent trimestre'.

[MCP] tool=get_employee args={'employee_id': 'E003'}
hr_server
get_employee(E003) -> {'name': 'Karim Bennani', ...}

[MCP] tool=add_note args={'employee_id': 'E003', 'note': 'Excellent
trimestre'}
notes_server
add_note('E003', 'Excellent trimestre')

Reponse finale : Karim Bennani (E003, Sales) - note "Excellent
trimestre" ajoutee.
```

Criteres de validation

`dual_agent.py` découvre bien les **5 outils** (3 HR + 2 notes).
 Le middleware affiche `[MCP] tool=...` avant chaque appel d'outil.
 Des outils de deux serveurs différents sont appelés pour répondre à la question.
 La réponse mentionne Karim Bennani (Sales) et confirme l'ajout de la note.

5 Pour aller plus loin (optionnel)

- **Durée d'exécution** : utiliser `time.time()` avant/après l'appel pour mesurer la latence.
- **Compteur global** : maintenir un compteur du nombre total d'appels d'outils par session.
- **RBAC simplifié** : bloquer `update_salary` si l'utilisateur n'a pas le rôle `hr_admin`.

6 Modalités de rendu

Déposez sur la plateforme un dossier zippé nommé `TP_MCP_NomPrenom.zip` contenant :

Fichier	Contenu
<code>math_server.py</code>	Exercice 1 (avec <code>divide</code> pour le bonus)
<code>math_agent.py</code>	Exercice 2
<code>notes_server.py</code>	Exercice 3
<code>hr_server.py</code>	Copie telle quelle depuis le cours (exercice 3)
<code>dual_agent.py</code>	Exercice 3
<code>trace.txt</code>	Sortie terminale de chaque exercice

Récapitulatif des concepts clés

Concept	Description	Exercice
FastMCP	Framework Python pour créer des serveurs MCP	1
@mcp.tool()	Décorateur exposant une fonction Python comme outil MCP	1
Inspector MCP	Outil de test isolé pour les serveurs (via <code>npx</code>)	1
MultiServerMCPClient	Client LangChain connectant un ou plusieurs serveurs	2, 3
ainvoke()	Invocation asynchrone obligatoire avec MCP	2, 3
asyncio.run()	Point d'entrée async dans un script Python	2, 3
Middleware	Décorateur wrappant les outils pour journalisation	3